

8. Oracle jet Common Model API/ collection Model API

8.1 Oracle JET - CRUD App Using oj-Model and oj-Table

Introduction

In this hands-on activity, you'll build a simple Oracle JET application that performs Create, Read, Update, and Delete (CRUD) operations using:

- oj-collection for data management
- oj-table to display data
- oj-form-layout and input components for interaction

Pre-requisites

- Oracle JET CLI installed (npm install -g @oracle/ojet-cli)
- JSON Server installed globally (npm install -g json-server)
- Basic knowledge of:
 - Knockout.js
 - Oracle JET component basics
 - MVVM architecture

What Are We Going to Do in This Practice?

In this hands-on practice, we will:

- Build a **User Management App** using Oracle JET.
- Use Oracle JET **form components** for user input.
- Use **oj-table** to display the list of users.
- Connect the app to a **mock REST API** using oj-collection and oj-model.
- Perform full **CRUD operations** (Create, Read, Update, Delete).
- Learn how to bind data dynamically using **Knockout observables**.

Why Do We Create a Mock API?

- We need an **API backend** to simulate a real server.

- Oracle JET `oj.Collection` and `oj.Model` expect REST-style endpoints (GET, POST, PATCH, DELETE).
- A mock API allows **testing without a real database**.
- It enables **faster prototyping** and testing during front-end development.

1. Mock API Setup (Created Independently of Project Folder)

- We create a separate `db.json` file that contains a sample array of user records.
- This file acts as a **fake database**.
- We use **JSON Server** to turn this file into a working REST API with endpoints like:
 - GET `/users`
 - POST `/users`
 - PATCH `/users/:id`
 - DELETE `/users/:id`

2. Create a folder named `jet-rest-backend` on cli

Mkdir jet-rest-backend

Cd jet-rest-backend

Notepad db.json

And write the following code into `db.json`

```
{
  "users": [
    {
      "id": "101",
      "name": "test1",
      "email": "test1@gmail.com"
    },
    {
      "id": "102",
      "name": "test2",
```

```
"email": "test2@gmail.com"
```

```
}}}
```

Running and Viewing the Mock Server

3. We run the JSON server in a separate terminal using: (**Note:** move to the jet-rest-backend folder and then run the json server)

```
json-server --watch db.json --port 3000
```

4. This will start a local server at:
<http://localhost:3000/users>
5. You can open this URL in a browser to **see and verify the data**.
6. Every time you add or update a user from the JET app, this file gets updated automatically.

This command will give below messages. This means mock server is up and running

```
PS D:\Courses\Oracle Jet\ojet labs\form-components\jet-rest-backend> json-server --watch db.json --port 3000
--watch/-w can be omitted, JSON Server 1+ watches for file changes by default
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching db.json...

(^ ^ ^ ^)

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/users
```

5. Create the project using the navdrawer template and name the project as “**crud-users-app**”. Run this project by running a command on terminal

ojet serve

6. This will automatically load the required library files for the navdrawer template and other project files.
7. Now create the project files . the location of the files are given with each code: (note : create models folder under js)

a. [Src/js/Models/userModel.js](#)

It Defines the **structure of a single user** and Sets the urlRoot to point to the mock API. Also, it Identifies the id field for CRUD operations.

```
define(['ojs/ojmodel'], function (oj) {
  const UserModel = oj.Model.extend({
```

```

    idAttribute: 'id',
    urlRoot: 'http://localhost:3000/users'
  });

  return UserModel;
});

```

b. Src/js/Models/userCollection.js

it Represents a **collection of user models**. It is Configured to fetch the entire list from /users endpoint and Used in the ViewModel for displaying and managing user records.

```

define(['ojs/ojmodel', 'models/userModel'], function (oj, UserModel) {
  const UserCollection = oj.Collection.extend({
    model: new UserModel,
    url: 'http://localhost:3000/users'
  });

  return new UserCollection();
});

```

c. Src/js/viewModels/crudusers.js

This is **core ViewModel** file. It Declares observables (id, name, email) for form input and Implements addUser, updateUser, deleteUser, and loadUsers logic. Also, Connects with the userCollection and feeds data to ArrayDataProvider.

```

define([
  'knockout',
  'models/userCollection',
  'ojs/ojarraydataprovder',
  'ojs/ojtable', 'ojs/ojinputtext', 'ojs/ojformlayout', 'ojs/ojbutton'
], function (ko, userCollection, ArrayDataProvider) {
  function ViewModel() {
    const self = this;

    self.users = userCollection;

    self.id = ko.observable('');
    self.name = ko.observable('');
    self.email = ko.observable('');

    self.dataProvider = ko.observable();
  }
});

```

```

        self.loadUsers = function () {
            self.users.fetch({
                success: function () {
                    self.dataProvider(new ArrayDataProvider(
self.users.map(function (model) {
                    return model.attributes;
                })), { keyAttributes: 'id' }
            ));
        },
        error: function () {
            alert('Failed to load users.');
        }
    });
};

self.addUser = function () {
    const existingUser = self.users.get(self.id());

    if (existingUser) {
        alert('User with this ID already exists!');
        return;
    }

    self.users.create({
        id: self.id(),
        name: self.name(),
        email: self.email()
    }, {
        wait: true,
        success: function () {
            self.loadUsers();
            alert('User added');
        },
        error: function () {
            alert('Failed to add user');
        }
    });
};

self.updateUser = function () {
    const user = self.users.get(self.id());
    if (user) {
        user.save({
            name: self.name(),
            email: self.email()
        }, {
            patch: true,
            success: function () {
                self.loadUsers();
            }
        });
    }
};

```

```

        alert('User updated');
    },
    error: function () {
        alert('Update failed');
    }
    });
}
};

self.deleteUser = function () {
    const user = self.users.get(self.id());
    if (user) {
        user.destroy({
            success: function () {
                self.loadUsers();
                alert('User deleted');
            },
            error: function () {
                alert('Delete failed');
            }
        });
    }
};

self.loadUsers();
}

return new ViewModel();
});

```

d. Src/js/view/crudusers.html

It Contains the **Oracle JET UI layout**. it Uses `<oj-form-layout>` for form fields and buttons. And Uses `<oj-table>` to display the user list from the data provider

```

<oj-form-layout>
  <oj-input-text label-hint="ID" value="{{id}}"></oj-input-text>
  <oj-input-text label-hint="Name" value="{{name}}"></oj-input-text>
  <oj-input-text label-hint="Email" value="{{email}}"></oj-input-text>

  <oj-button on-oj-action="[[addUser]]">Add</oj-button>

```

```

<oj-button on-oj-action="[[updateUser]]">Update</oj-button>
<oj-button on-oj-action="[[deleteUser]]">Delete</oj-button>
</oj-form-layout>

<oj-table
  data="[[dataProvider]]"
  columns='[
    {"headerText": "ID", "field": "id"},
    {"headerText": "Name", "field": "name"},
    {"headerText": "Email", "field": "email"}
  ]'
  display="grid"
  class="oj-sm-12">
</oj-table>

```

e. Define the route to crudusers through appController.js file

```

let navData = [
  { path: '', redirect: 'dashboard' },
  { path: 'dashboard', detail: { label: 'Dashboard', iconClass: 'oj-ux-ico-bar-chart' } },
  { path: 'incidents', detail: { label: 'Incidents', iconClass: 'oj-ux-ico-fire' } },
  { path: 'customers', detail: { label: 'Customers', iconClass: 'oj-ux-ico-contact-group' } },
  { path: 'about', detail: { label: 'About', iconClass: 'oj-ux-ico-information-s' } },
  { path: 'crudusers', detail: { label: 'CRUD Users', iconClass: 'oj-ux-ico-contact-group' } }
];

```

8. Run the web app

Ojet serve

ORACLE App Name john.hancock@oracle.com

Dashboard Incidents Customers About CRUD Users

103

test3

test3@gmail.com

Add

Update

Delete

ID	Name	Email
101	test1	test1@gmail.com
102	test2	test2@gmail.com
103	test3	test3@gmail.com

Your UI might be different because I have already made some random entries for testing.

Now test for add :

Fill the ID, name and Email and click on add button.. this will show the content added in the last row of the table. If you try to add user with same id , this will give error as below screenshot

The screenshot shows a web application interface. At the top left is the text "ORACLE App Name". At the top right is the user "john.hancock@oracle.com" with a dropdown arrow. Below the user name are navigation links: "Incidents", "Customers", "About", and "CRUD Users". The main form area contains three input fields: "ID" with the value "103", "Name" with the value "test3", and "Email" with the value "test3@gmail.com". Below these fields are three buttons: "Add", "Update", and "Delete". A modal error message is displayed in the center, stating "localhost:8000 says" and "User with this ID already exists!". At the bottom is a table with three columns: "ID", "Name", and "Email". The table contains three rows of data.

ID	Name	Email
101	test1	test1@gmail.com
102	test2	test2@gmail.com
103	test3	test3@gmail.com

Test for update:

Fill the id that exists already and update the other fields. Then click on update button.

Test for delete

Fill the already existed id, name and email and click on delete.

To check whether it is adding, updating and deleting users, see the contents db.json file

JS userModel.js U

JS userCollection.js U

JS crudusers.js U

jet-rest-backend > {} db.json > ...

```
1  {
2    "users": [
3      {
4        "id": "101",
5        "name": "test1",
6        "email": "test1@gmail.com"
7      },
8      {
9        "id": "102",
10       "name": "test2",
11       "email": "test2@gmail.com"
12     },
13     {
14       "id": "103",
15       "name": "test3",
16       "email": "test3@gmail.com"
17     }
18   ]
19 }
```